

bot 指令集

文档变更记录

日期	版本	操作内容	操作人
25. 07. 11	V1.0	协议文档定义	于海生

通用渲染指令

1. RenderVoiceInputText 指令

设备端在进行 ListenStarted 事件请求时，如果服务端识别到用户说话的内容，则会把识别到内容下发 RenderVoiceInputText 指令到设备端。

消息样例蓝牙

JSON

```
"directive": {
  "header": {
    "namespace": "ai.dueros.device_interface.screen",
    "name": "RenderVoiceInputText",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "text": "{{STRING}}",
    "type": "{{STRING}}",
    "voiceInputMode": "{{ENUM}}",
    "rolling": {{BOOLEAN}}
  }
}
```

Payload 参数说明

- text

- 语音识别的结果
- type
 - 用于表示 query 的状态
 - 取值
 - . "INTERMEDIATE": 中间结果
 - . "FINAL": 最终结果
 - . "REJECTED": 在全双工交互模式下，服务端识别后最终认为不需要处理，比如：环境噪音触发的识别等
- voiceInputMode
 - 当前语音输入所处的模式
 - 取值
 - . "WAKE_UP": 处在唤醒后的第一轮收音模式
 - . "FOLLOW_UP": 处在全双工的 FOLLOW_UP 收音模式下
 - . "SERVER_EXPECT_VOICE_INPUT": 处在服务端期待用户输入的模式下
- rolling
 - 控制上屏是否滚动显示减小上屏占用宽度，默认值为 false。
 - 取值
 - . true: 滚动显示
 - . false: 非滚动显示

2. RenderCard 指令

提供 4 种卡片，在有屏幕设备上可以渲染 UI 界面

卡片名	描述	使用实例
TextCard	只能显示文字、链接	聊天，文本笑话
StandardCard	能显示文字、链接、一张图片	带图片的百科介绍
ListCard	能显示列表项，每个列表项可以显示文字、链接、一张图片	电影介绍、美食介绍
ImageListCard	只能显示多张图片	图片服务、图片笑话

点击图片可查看完整电子表格

2.1 TextCard

JSON

```
"directive": {
  "header": {
    "namespace": "ai.dueros.device_interface.screen",
    "name": "RenderCard",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "token": "{{STRING}}",
    "type": "TextCard",
    "content": "{{STRING}}",
    "link": {{LinkStructure}}
  }
}
```

- content
 - 展现的文字
- (optional) link
 - 底部链接

2.2 StandardCard

JSON

```
"directive": {
  "header": {
    "namespace": "ai.dueros.device_interface.screen",
    "name": "RenderCard",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "token": "{{STRING}}",
    "type": "StandardCard",
    "image": {{ImageStructure}},
    "title": "{{STRING}}",
    "content": "{{STRING}}",
    "link": {{LinkStructure}}
  }
}
```

- (optional) image

- 图片地址
- title
 - 标题，建议稍大字号，加粗显示
- content
 - 摘要信息，能多行显示
- (optional) link
 - 底部链接

2.3 ListCard

JSON

```
"directive": {
  "header": {
    "namespace": "ai.dueros.device_interface.screen",
    "name": "RenderCard",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "token": "{{STRING}}",
    "type": "ListCard",
    "title": "{{STRING}}",
    "list": [
      {
        "title": "{{STRING}}",
        "content": "{{STRING}}",
        "image": {{ImageStructure}},
        "url": "{{STRING}}"
      },
      {
        "title": "{{STRING}}",
        "content": "{{STRING}}",
        "image": {{ImageStructure}},
        "url": "{{STRING}}"
      },
      ...
    ],
    "link": {{LinkStructure}}
  }
}
```

(optional) title

- 列表的标题
- list
 - 可以有多个 item
- list[].title
 - 该列表 item 的标题
- list[].content
 - 该列表 item 的摘要，能显示多行
- (optional) list[].image
 - 图片
- (optional) list[].url
 - 点击该列表项，跳转的地址
- (optional) link
 - 整体卡片的链接

2.4 ImageListCard

```
JSON
{
  "directive": {
    "header": {
      "namespace": "ai.dueros.device_interface.screen",
      "name": "RenderCard",
      "messageId": "{{STRING}}",
      "dialogRequestId": "{{STRING}}"
    },
    "payload": {
      "token": "{{STRING}}",
      "type": "ImageListCard",
      "imageList": [
        {{ImageStructure}},
        {{ImageStructure}},
        ...
      ]
    }
  }
}
```

- imageList
 - 图片列表

2.5 ImageStructure 结构

```
JSON
{
  "originPageUrl": "{{STRING}}",
  "src": "{{STRING}}",
  "size": "{{STRING}}"
}
```

- src
 - 图片地址
- (optional) originPageUrl
 - 图片来源地址(可能是网页地址, 或者网站来源主页)
- (optional) size
 - 图片的尺寸, 样例值: 200*200

2.6 LinkStructure 结构

```
JSON
{
  "url": "{{STRING}}",
  "anchorText": "{{STRING}}"
}
```

- url
 - 链接地址
- anchorText
 - 锚文本

3. RenderStreamCard 指令

上屏指令, 用于流式回复渲染上屏文字, 通常配合 **Speak** 指令同时下发

消息样例

```
json
"directive": {
  "header": {
    "namespace": "ai.dueros.device_interface.screen",
```

```

    "name": "RenderStreamCard",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "id": "{{STRING}}",
    "answer": "北京特产有很多...",
    "part": "造工艺而著名。",
    "index": 18,
    "is_end": 0
  }
}

```

Payload 参数说明

- id: 当前 CardId, 需要根据此 id 来做流式展示
- answer: 完整的回复内容
- part: 增量的回复内容
- index: 当前第几个分段, 从 0 开始
- isEnd: 是否为最后一个分片, 是为 1、否则为 0

4. RenderHint 指令

渲染引导词。引导词是给用户的提示, 提示用户当前可以输入哪些 query

消息样例

```

JSON
"directive": {
  "header": {
    "namespace": "ai.dueros.device_interface.screen",
    "name": "RenderHint",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "token": "{{STRING}}",
    "cueWords": [
      "{{STRING}}",
      "{{STRING}}",
      ...
    ]
  }
}

```

```
]
}
}
```

Payload 参数说明

- cueWords
 - 列表，可以有多个引导词

语音播报指令

摘要

本接口定义语音输出相关的功能，如果设备有扬声器，则应该实现本接口，接入语音输出的能力。

1. Speak 指令

在服务端需要通过播报来满足用户请求时，服务端会下发 **Speak** 指令，附带需要播报的音频内容。设备端收到 **Speak** 指令，应该播报返回中所附带的音频内容。

消息样例

```
JSON
{
  "directive": {
    "header": {
      "namespace": "ai.dueros.device_interface.voice_output",
      "name": "Speak",
      "messageId": "{{STRING}}",
      "dialogRequestId": "{{STRING}}"
    },
    "payload": {
      "token": "{{STRING}}",
      "format": "{{STRING}}",
      "url": "{{STRING}}",
      "content": "{{STRING}}"
    }
  }
}
```

Payload 参数说明

- token
 - 该播报对应的唯一 token
- format
 - 所附带播报的数据格式
 - 取值
 - AUDIO_MPEG: mp3 格式
 - TEXT: 文本内容
- url
 - format=AUDIO_MPEG 时，为语音链接地址
- content
 - format=TEXT 时，为原始语音文本

语音输入指令

摘要

本接口定义语音输入相关的功能，如果设备有麦克风，应该实现本接口，接入语音输入的能力。接口包括设备端进行语音请求（上传语音），服务端下发开始收听语音、停止收听语音指令等等。

1. StopListen 指令

设备端在进行 ListenStarted 事件请求时，如果服务端检测到 VAD（Voice Activity Detection，用来检测用户是不是说完了），则会下发 StopListen 指令到设备端。

设备端在收到 StopListen 指令时，应该立即停止收听语音，并关闭麦克风。

消息样例

```
JSON
{
  "directive": {
    "header": {
      "namespace": "ai.dueros.device_interface.voice_input",
      "name": "StopListen",
      "messageId": "{{STRING}}",
      "dialogRequestId": "{{STRING}}"
    },
    "payload": {}
  }
}
```

```
"payload": {  
  }  
}
```

2. Listen 指令

在对话当中，如果服务端需要用户提供更多信息，需要通过多轮对话来询问，则服务端在处理完当前语音请求之后，下发 **Listen** 指令到设备端，主动发起下一轮对话。

设备端在执行 **Listen** 指令时，应该立即开启麦克风，开始收听用户语音请求，具体的执行方法参考下面 **type** 参数的描述。

消息样例

```
JSON  
"directive": {  
  "header": {  
    "namespace": "ai.dueros.device_interface.voice_input",  
    "name": "Listen",  
    "messageId": "{{STRING}}",  
    "dialogRequestId": "{{STRING}}"  
  },  
  "payload": {  
    "timeoutInMilliseconds": {{LONG}},  
    "duplexListenTimeoutInMilliseconds": {{LONG}},  
    "followUpAskTimeoutInMilliseconds": {{LONG}}  
  }  
}
```

Payload 参数说明

- **timeoutInMilliseconds**
 - 设备端执行 **Listen** 指令后，等待麦克风打开将录音发送给云端，如果在指定的时间内(**timeoutInMilliseconds**)麦克风没有打开，设备端需要上报 **ListenTimeOut** 事件。执行 **Listen** 指令后，麦克风没有打开的情况一般出现在需要用户按键来打开麦克风的设备上，即 **PRESS_AND_HOLD** 类型的语音输入设备。
- (optional) **duplexListenTimeoutInMilliseconds**
 - 全双工模式下，在服务端通过 **Listen** 指令打开麦克风时，设置麦克风的收音时长，参数效果与 **SetDuplexListenTimeout** 指令相似；对于非全双工模式，此参数无效；

- 如果此参数不存在，而 `initiator.type` 为 `FOLLOW_UP` 时，聆听超时缺省设置为 30 秒
- (optional) `followUpAskTimeoutInMilliseconds`
 - 在全双工交互场景下，如果设备已经处在聆听状态，又收到服务端下发的 `Listen` 指令，那么在 `followUpAskTimeoutInMilliseconds` 时间内，没有有效的用户输入，上报 `FollowUpAskListenTimedOut` 事件

音频播放指令

摘要

本接口提供了播放音频相关的指令和事件。通过本接口定义的指令可以控制设备端上音频资源的播放，并通过事件可以感知播放进度和缓冲区状态等信息。为了更好的理解这个接口，我们先来看看 `Audio Player` 有哪些状态：

- **PLAYING**：设备端正在播放 `audio item` 时所处的状态。**PLAYING** 状态时可向服务器发送播放进度和 `audio item` 的元数据。以下几种情况会退出 **PLAYING** 状态：
 - 收到暂停或停止音频流的指令
 - 音频流缓冲失败
 - 播放失败
- **STOPPED**：以下几种情况会处于 **STOPPED** 状态：
 - 音频流出现问题导致播放失败
 - 服务端下发 `STOP` 指令
 - 收到 `ClearQueue` 指令，其中 `clearBehavior` 参数值为 `CLEAR_ALL`
 - 收到 `Play` 指令，其中 `playBehavior` 参数值为 `REPLACE_ALL`
 - 处于 **PAUSED** 或 **BUFFER_UNDERRUN** 状态时，收到 `clearBehavior` 参数值为 `CLEAR_ALL` 的 `ClearQueue` 指令
- **PAUSED**：当优先级更高的 `Channel` 进入活跃状态时（例如用户开始语音交互、闹铃响起），应当进入 **PAUSED** 状态。
- **BUFFER_UNDERRUN**：当设备端处于 **PLAYING** 状态，并且缓冲的数据不足以继续播放，播放器等待缓冲数据时进入 **BUFFER_UNDERRUN** 状态，直到缓冲完成才会恢复为 **PLAYING** 状态。
- **FINISHED**：在 `audio item` 播放完成时将转为 **FINISHED** 状态。即使设备端本地播放队列中有多个 `audio item` 待播放，在每一个 `audio item` 播放结束后也都需要向服

务端发送 PlaybackFinished 事件，并且进入 FINISHED 状态。另外，设备启动后的初始状态也应当是 FINISHED 状态。

- **IDLE**: 端上播放器没有被任何音频占用
 - 无屏端场景: 端上首次进入音频播放前，状态为 IDLE，一旦播放过音频后，在关机断电前一直处于非 IDLE 状态。
 - 有屏端场景: 端上前后台都没有显示播放器的情况下，认定为 IDLE 状态；区别于 STOPPED，STOPPED 表明停止播放，但是播放器还在前台或者后台，用户可见的情况
 - 无 PlaybackState 状态上报的情况下，等价于 IDLE 状态

1. Play 指令

Play 指令控制设备端的音频播放，指令包含一个所要播放的 audio item，以及控制播放行为的 playBehavior 参数。

控制设备端播放和队列的 playBehavior 参数，有以下几种类型：

- **REPLACE_ALL**: 立即停止当前播放（如有必要，发送 PlaybackStopped 事件）并清除播放队列，立即播放指令中的 audio item。
- **ENQUEUE**: 将 audio item 添加到当前队列的尾部。
- **REPLACE_ENQUEUED**: 替换播放队列中的所有 audio item，但不影响当前正在播放的 audio item。

audio item 还包含 expectedPreviousToken 参数，该参数的处理有以下几种情况：

- 如果 expectedPreviousToken 不存在，则 Play 指令正常执行
- 在 playBehavior 为 ENQUEUE 时，expectedPreviousToken 应该与当前播放队列末尾 audio item 中的 token 一致，如果不一致则不执行本 Play 指令
- 在 playBehavior 为 REPLACE_ENQUEUED 时，expectedPreviousToken 应该与当前正在播放的 audio item 中的 token 一致，如果不一致则不执行本 Play 指令
- playBehavior 为 REPLACE_ALL 时，expectedPreviousToken 不存在

消息样例

JSON

```
"directive": {
  "header": {
    "namespace": "ai.dueros.device_interface.audio_player",
    "name": "Play",
```

```

        "messageId": "{{STRING}}",
        "dialogRequestId": "{{STRING}}"
    },
    "payload": {
        "playBehavior": "{{STRING}}",
        "audioItem": {
            "audioItemId": "{{STRING}}",
            "extension": "{{STRING}}",
            "stream": {
                "url": "{{STRING}}",
                "streamFormat": "AUDIO_MPEG",
                "speed": "{{ENUM}}",
                "offsetInMilliseconds": {{LONG}},
                "expiryTime": "{{STRING}}",
                "progressReport": {
                    "progressReportDelayInMilliseconds": {{LONG}},
                    "progressReportIntervalInMilliseconds":
{{LONG}}
                },
                "token": "{{STRING}}",
                "expectedPreviousToken": "{{STRING}}",
                "chorus": {
                    "onlyChorus": {{BOOLEAN}},
                    "startInMilliseconds": {{LONG}},
                    "endInMilliseconds": {{LONG}}
                }
            }
        }
    }
}

```

Payload 参数说明

- playBehavior
 - 控制客户端播放和队列处理，详见前文说明
- audioItem
 - 所要播放的音频资源
- audioItem.audioItemId
 - audio item 的 id
- (optional) audioItem.extension
 - 扩展信息，Base64 格式字符串，存放的是一个 json，云端根据需要进行设置，该字段会被设置到 clientContext 中

- `audioItem.stream`
 - 音频流的元信息
- `audioItem.stream.url`
 - 音频流的资源地址。如果是外部音频资源，则为 `http/https` 链接；如果为 `"cid:"`，则本 `http` 回复附件中附带音频流
- `audioItem.stream.streamFormat`
 - 如果音频流为 `http` 回复附件时，表示音频流的格式（目前支持的格式为 `AUDIO_MPEG`）；如果音频流为外部 `url` 时此字段不存在
- (optional) `audioItem.stream.speed`
 - 播放倍速信息。取值: `0.5,0.75,1,1.25,1.5,2` (不支持倍速可不输出此字段)
- `audioItem.stream.offsetInMilliseconds`
 - 从指定的 `offset` 开始进行播放
- `audioItem.stream.expiryTime`
 - `ISO8601` 格式，表示 `stream` 过期时间
- `audioItem.stream.progressReport.progressReportDelayInMilliseconds`
 - 如果此字段存在，则设备端在播放该 `audio item` 时，播放到所指定时间之后应该上报 `ProgressReportDelayElapsed` 事件；如果此字段不存在，则设备端端不需要上报 `ProgressReportDelayElapsed` 事件
- `audioItem.stream.progressReport.progressReportIntervalInMilliseconds`
 - 如果此字段存在，则设备端在播放该 `audio item` 时，每隔指定时间上报 `ProgressReportIntervalElapsed` 事件；如果此字段不存在，则设备端不需要上报 `ProgressReportIntervalElapsed` 事件
- `audioItem.stream.token`
 - 代表本 `audio item` 的唯一 `token`
- `audioItem.stream.expectedPreviousToken`
 - 如果此字段存在，则应当匹配前一个 `audio item` 中的 `token`，如果不匹配则不执行本 `Play` 指令；详见前文说明
- (optional) `audioItem.stream.chorus`
 - 歌曲的副歌信息，如果有该字段表示本首歌曲有副歌;场景电台付费试听下发字段
- (optional) `audioItem.stream.chorus.onlyChorus`
 - 是否只能播放副歌，如果该字段为 `true`，则播放器不能播放副歌以外的部分
- (optional) `audioItem.stream.chorus.startInMilliseconds`

- 副歌的起始时间点;场景电台试听起始时间点
- (optional) `audioItem.stream.chorus.endInMilliseconds`
 - 副歌的结束时间点;场景电台试听起始时间点

2. SetPlaybackSpeed 指令

用于控制音频播放时的倍速变化

消息样例

JSON

```
"directive": {
  "header": {
    "namespace": "ai.dueros.device_interface.audio_player",
    "name": "SetPlaybackSpeed",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "speed": "{{ENUM}}"
  }
}
```

Payload 参数说明

- speed:
 - 倍速数值
 - 取值: 0.5,0.75,1,1.25,1.5,2 (端支持速率)

3. PlaybackStarted 事件

audio item 开始播放时上报此事件。

注意：对每一个服务端下发的 URL（就是 **Play** 指令里面下发的 URL），设备端应该只上报一次 **PlaybackStarted** 事件。如果设备端收到的是一个播放列表的 URL，也应该只上报一个 **PlaybackStarted** 事件。

消息样例

JSON

```
"event": {
  "header": {
```

```
    "namespace": "ai.dueros.device_interface.audio_player",
    "name": "PlaybackStarted",
    "messageId": "{{STRING}}"
  },
  "payload": {
    "token": "{{STRING}}",
    "offsetInMilliseconds": {{LONG}},
    "playerName": "{{STRING}}",
    "linkFrom": "{{STRING}}"
  }
}
```

Payload 参数说明

- token:
 - 要播放的 audio item 的 token
- offsetInMilliseconds
 - 要开始播放的 offset
- (optional)playerName
 - 播放器的类型，可选字段，如果不上报此字段，则默认当前事件上报来自音频播放器
 - SIMPLIFY_MODE 新增的 30s 听歌模式专用播放器
- (optional)linkFrom
 - 链接来源

4. PlaybackFinished 事件

audio item 播放结束时上报此事件。

以下情况下不上报本事件：

- 播放停止（Stop 指令，或者设备本地停止按钮）
- 上一首下一首切换时

注意：对每一个服务器下发的 URL（就是 Play 指令里面下发的 URL），设备端应该只上报一次 **PlaybackFinished** 事件。如果设备端收到的是一个播放列表的 URL，也应该只上报一个 **PlaybackFinished** 事件。

消息样例


```

JSON
"clientContext": [
  {{ai.dueros.device_interface.settings.SettingsState}}
],
"event": {
  "header": {
    "namespace": "ai.dueros.device_interface.audio_player",
    "name": "PlaybackFinished",
    "messageId": "{{STRING}}"
  },
  "payload": {
    "token": "{{STRING}}",
    "offsetInMilliseconds": {{LONG}},
    "playerName": "{{STRING}}",
    "linkFrom": "{{STRING}}"
  }
}

```

Client Context 说明

需要上报 ai.dueros.device_interface.audio_player.SettingsState，是因为云端召回下一首歌时，需要用到 SettingsState 中，状态以客户端为准的设置，比如 qq 音乐 app 的开关等。

Payload 参数说明

- token
 - 播放完成的 audio item 的 token
- offsetInMilliseconds
 - 播放完成时的 offset
- (optional)playerName
 - 播放器的类型，可选字段，如果不上报此字段，则默认当前事件上报来自音频播放器
 - SIMPLIFY_MODE 新增的 30s 听歌模式专用播放器
- (optional)linkFrom
 - 链接来源

5. PlaybackNearlyFinished 事件

当当前 audio item 播放快要结束，准备缓冲/下载播放队列中的下一个 audio item 时上报此事件。服务端有可能会返回 Play 指令，添加新的 audio item 到队列末尾。

消息样例

JSON

```
"clientContext": [
  {{ai.dueros.device_interface.settings.SettingsState}}
],
"event": {
  "header": {
    "namespace": "ai.dueros.device_interface.audio_player",
    "name": "PlaybackNearlyFinished",
    "messageId": "{{STRING}}"
  },
  "payload": {
    "token": "{{STRING}}",
    "offsetInMilliseconds": {{LONG}},
    "playerName": "{{STRING}}",
    "linkFrom": "{{STRING}}"
  }
}
```

Client Context 说明

需要上报 ai.dueros.device_interface.settings.SettingsState，是因为云端召回下一首歌时，需要用到 SettingsState 中，状态以客户端为准的设置，比如 qq 音乐 app 的开关等。

Payload 参数说明

- token
 - 本 audio item 的 token
- offsetInMilliseconds
 - 当前的播放进度
- (optional)playerName
 - 播放器的类型，可选字段，如果不上报此字段，则默认当前事件上报来自音频播放器
 - SIMPLIFY_MODE 新增的 30s 听歌模式专用播放器
- (optional)linkFrom

- 链接来源

6. PlaybackFailed 事件

当设备端播放 audio item 发生错误时上报此事件。

事件中的 token 有可能与正在播放中的 audio item 的 token 不一致，比如正在播放一个 audio item，并要开始缓冲下一个 audio item 时下一个 audio item 的缓冲出现错误。

消息样例

```
JSON
{
  "clientContext": [
    {{ai.dueros.device_interface.audio_player.PlaybackState}},
    {{ai.dueros.device_interface.settings.SettingsState}}
  ],
  "event": {
    "header": {
      "namespace": "ai.dueros.device_interface.audio_player",
      "name": "PlaybackFailed",
      "messageId": "{{STRING}}"
    },
    "payload": {
      "token": "{{STRING}}",
      "_mediaHostInfo": {
        "domainName": "{{STRING}}",
        "ipAddress": "{{STRING}}"
      },
      "error": {
        "type": "{{STRING}}",
        "message": "{{STRING}}"
      },
      "playerName": "{{STRING}}"
    }
  }
}
```

Client Context 说明

本事件需要设备端上报端上的 ai.dueros.device_interface.audio_player.PlaybackState 状态。

需要上报 ai.dueros.device_interface.audio_player.SettingsState，是因为云端召回下一首歌时，需要用到 SettingsState 中，状态以客户端为准的设置，比如 qq 音乐 app

的开关等。

Payload 参数说明

- token
 - 发生错误的 audio item 的 token
- (optional) _mediaHostInfo
 - 正在播放的音频资源的域名、IP 地址等信息，用于辅助定位播放失败原因
- _mediaHostInfo.domainName
 - 正在播放的音频资源的域名
- _mediaHostInfo.ipAddress
 - 正在播放的音频资源的 IP 地址，由于每次域名解析的 IP 地址可能不同，这里的 IP 地址是指音频资源在此次播放时播放器所解析出的 IP 地址
- error.type
 - 错误类型，取值如下
 - "MEDIA_ERROR_UNKNOWN": 未知错误
 - "MEDIA_ERROR_INVALID_REQUEST": stream 服务端返回请求无效 (可能的情况有 bad request, unauthorized, forbidden, not found 等等)
 - "MEDIA_ERROR_SERVICE_UNAVAILABLE": 设备端无法连接 stream 服务端
 - "MEDIA_ERROR_INTERNAL_SERVER_ERROR": stream 服务端接受请求，但未能正确处理
 - "MEDIA_ERROR_INTERNAL_DEVICE_ERROR": 设备端内部错误
- error.message:
 - 错误描述，仅用于打印日志；如果有 HTTP 相关的错误，也应该包含在本错误消息中，以便排查问题
- (optional)playerName
 - 播放器的类型，可选字段，如果不上报此字段，则默认当前事件上报来自音频播放器
 - SIMPLIFY_MODE 新增的 30s 听歌模式专用播放器

7. ProgressReportDelayElapsed 事件

本事件用来统计播放情况。如果 Play 指令中有 progressReportDelayInMilliseconds,

则对应 audio item 播放此时间长后需要上报本事件。

注意一、不是播放 offset 到此时间，而是播放音频此时间长。例如，progressReportDelayInMilliseconds 为 10000，Play 指令指定的 offset 为 25000，则音频从第 25 秒处开始播放，当播放了 10 秒钟之后（播放到 offset 为 35 秒处），上报本事件。

注意二、设置倍速播放 speed 后，上报周期要以物理时间为准，而不是播放进度条为准

消息样例

JSON

```
"event": {
  "header": {
    "namespace": "ai.dueros.device_interface.audio_player",
    "name": "ProgressReportDelayElapsed",
    "messageId": "{{STRING}}"
  },
  "payload": {
    "token": "{{STRING}}",
    "offsetInMilliseconds": {{LONG}},
    "playerName": "{{STRING}}"
  }
}
```

Payload 参数说明

- token
 - 本 audio item 的 token
- offsetInMilliseconds
 - 当前的播放进度
- (optional)playerName
 - 播放器的类型，可选字段，如果不上报此字段，则默认当前事件上报来自音频播放器
 - SIMPLIFY_MODE 新增的 30s 听歌模式专用播放器

8. ProgressReportIntervalElapsed 事件

本事件用来统计播放情况。如果 Play 指令有 progressReportIntervalInMilliseconds，则在播放对应 audio item 时，每隔此时间上报本事件。

注意，上报本事件应根据播放时长，而不是播放 **offset**。例如，**progressReportIntervalInMilliseconds** 为 10000，**Play** 指令指定的 **offset** 为 25000，则音频从第 25 秒开始播放，每播放 10 秒钟（播放到 **offset** 为 35 秒处、45 秒处、55 秒处等等），上报本事件。

消息样例

```
JSON
{
  "event": {
    "header": {
      "namespace": "ai.dueros.device_interface.audio_player",
      "name": "ProgressReportIntervalElapsed",
      "messageId": "{{STRING}}"
    },
    "payload": {
      "token": "{{STRING}}",
      "offsetInMilliseconds": {{LONG}},
      "playerName": "{{STRING}}"
    }
  }
}
```

Payload 参数说明

- token
 - 本 audio item 的 token
- offsetInMilliseconds
 - 当前的播放进度
- (optional)playerName
 - 播放器的类型，可选字段，如果不上报此字段，则默认当前事件上报来自音频播放器
 - SIMPLIFY_MODE 新增的 30s 听歌模式专用播放器

9. PlaybackStutterStarted 事件

在 PlaybackStarted 事件之后，并且设备端播放器已经在开始播放资源过程中，如果资源播放出现卡顿，则立即上报此事件。事件上报后音频播放器应该进入 BUFFER_UNDERRUN 状态，并保持此状态直到缓冲区满并恢复音频播放。

判断播放卡顿可供参考的方法

- 播放器缓存中没有音频数据可以继续播放，即可上报该事件

- 播放器从开始播放到当前的物理时间间隔，大于播放器播放进度时长 2s 以上，即可上报该事件

消息样例

JSON

```
"event": {
  "header": {
    "namespace": "ai.dueros.device_interface.audio_player",
    "name": "PlaybackStutterStarted",
    "messageId": "{{STRING}}"
  },
  "payload": {
    "token": "{{STRING}}",
    "offsetInMilliseconds": {{LONG}},
    "_mediaHostInfo": {
      "domainName": "{{STRING}}",
      "ipAddress": "{{STRING}}"
    },
    "error": {
      "message": "{{STRING}}"
    },
    "playerName": "{{STRING}}"
  }
}
```

Payload 参数说明

- token
 - 本 audio item 的 token
- offsetInMilliseconds
 - 当前的播放进度
- (optional) _mediaHostInfo
 - 正在播放的音频资源的域名、IP 地址等信息，用于辅助定位缓冲导致的播放卡顿原因
- _mediaHostInfo.domainName
 - 正在播放的音频资源的域名
- _mediaHostInfo.ipAddress
 - 正在播放的音频资源的 IP 地址，由于每次域名解析的 IP 地址可能不同，这里的 IP 地址是指音频资源在此次播放时播放器所解析出的 IP 地址

- `error.message`:
 - 错误描述，仅用于打印日志；如果有 HTTP 相关的错误，也应该包含在本错误消息中，以便排查问题
- (optional)`playerName`
 - 播放器的类型，可选字段，如果不上报此字段，则默认当前事件上报来自音频播放器
 - SIMPLIFY_MODE 新增的 30s 听歌模式专用播放器

10. PlaybackStutterFinished 事件

PlaybackStutterStarted 事件之后，缓冲恢复到正常状态，可重新开始播放时（播放资源卡顿恢复）上报本事件；重新开始播放时，不需要再额外上报 PlaybackStarted 事件。

注意：本事件可能不会同 **PlaybackStutterStarted** 事件配对出现。

消息样例

```
JSON
{
  "event": {
    "header": {
      "namespace": "ai.dueros.device_interface.audio_player",
      "name": "PlaybackStutterFinished",
      "messageId": "{{STRING}}"
    },
    "payload": {
      "token": "{{STRING}}",
      "offsetInMilliseconds": {{LONG}},
      "stutterDurationInMilliseconds": {{LONG}},
      "playerName": "{{STRING}}"
    }
  }
}
```

Payload 参数说明

- `token`
 - 本 audio item 的 token
- `offsetInMilliseconds`
 - 当前的播放进度

- stutterDurationInMilliseconds
 - 从 PlaybackStutterStarted 到 PlaybackStutterFinished 时间间隔
- (optional)playerName
 - 播放器的类型，可选字段，如果不上报此字段，则默认当前事件上报来自音频播放器
 - SIMPLIFY_MODE 新增的 30s 听歌模式专用播放器

11. Stop 指令

停止音频播放。用户的语音指令，用户按下设备端物理按钮，或者通过软件界面点击等情况都有可能使得服务端下发 Stop 指令。

消息样例

```
JSON
{"directive": {
  "header": {
    "namespace": "ai.dueros.device_interface.audio_player",
    "name": "Stop",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
  }
}}
```

12. PlaybackStopped 事件

停止音频播放时上报，有以下几种可能：

- 收到 Stop 指令，停止了播放
- 收到了 Play 指令，playBehavior 为 REPLACE_ALL，停止了当前音频流的播放
- 收到 ClearQueue 指令，clearBehavior 为 CLEAR_ALL，停止了当前音频流的播放
- 对于有停止播放按键（包括暂停播放按键）的设备，用户点击停止播放按键，停止了当前音频流的播放
- 对于有屏设备，用户点击屏幕上的停止播放按钮（包括暂停按钮），停止了当前音频流的播放

- 其他一些指令的执行或用户操作导致停止了当前音频流的播放，并且在指令执行完或者用户操作完毕后不再恢复当前音频流播放的情况，比如执行 `extensions.screen_navigation` 端能下的 `Exit` 指令时，退出 `audio_player` 并停止了当前音频流的播放，或者用户点击退出按钮退出了播放器等情况。补充说明一下，指令执行完恢复当前音频流播放的典型情况是音乐播放被语音播报打断的情况，这时应该上报 `PlaybackPaused/PlaybackResumed`，具体可以参见 `PlaybackPaused/PlaybackResumed` 事件的描述。

注意：本事件的上报，是在上述情况下停止了当前音频流的播放时发生。如果音频播放完毕正常结束，应该上报 **`PlaybackFinished`** 事件，不上报本事件。

消息样例

```
JSON
"event": {
  "header": {
    "namespace": "ai.dueros.device_interface.audio_player",
    "name": "PlaybackStopped",
    "messageId": "{{STRING}}"
  },
  "payload": {
    "token": "{{STRING}}",
    "offsetInMilliseconds": {{LONG}},
    "playerName": "{{STRING}}"
  }
}
```

Payload 参数说明

- `token`
 - 本 audio item 的 token
- `offsetInMilliseconds`
 - 当前的播放进度
- (optional)`playerName`
 - 播放器的类型，可选字段，如果不上报此字段，则默认当前事件上报来自音频播放器
 - `SIMPLIFY_MODE` 新增的 30s 听歌模式专用播放器

13. PlaybackPaused 事件

在音频播放时，如果发生对话/闹钟等更高优先级的 Channel 行为，则音频播放应该暂停，等更高优先级的 Channel 行为结束之后再重新继续播放。此时，应该上报 PlaybackPaused 事件和 PlaybackResumed 事件。

注意：如果发生语音输入，应该先上报 **ListenStarted** 事件，之后再上报 **PlaybackPaused** 事件，以减少语音交互延迟。

消息样例

```
JSON
"event": {
  "header": {
    "namespace": "ai.dueros.device_interface.audio_player",
    "name": "PlaybackPaused",
    "messageId": "{{STRING}}"
  },
  "payload": {
    "token": "{{STRING}}",
    "offsetInMilliseconds": "{{STRING}}",
    "playerName": "{{STRING}}"
  }
}
```

Payload 参数说明

- token
 - 本 audio item 的 token
- offsetInMilliseconds
 - 当前的播放进度
- (optional)playerName
 - 播放器的类型，可选字段，如果不上报此字段，则默认当前事件上报来自音频播放器
 - SIMPLIFY_MODE 新增的 30s 听歌模式专用播放器

14. PlaybackResumed 事件

在音频播放时，如果发生对话/闹钟等更高优先级的 Channel 行为，则音频播放应该暂停，等更高优先级的 Channel 行为结束之后再重新继续播放。此时，应该上报 PlaybackPaused 事件和 PlaybackResumed 事件。

消息样例

JSON

```
"event": {
  "header": {
    "namespace": "ai.dueros.device_interface.audio_player",
    "name": "PlaybackResumed",
    "messageId": "{{STRING}}"
  },
  "payload": {
    "token": "{{STRING}}",
    "offsetInMilliseconds": "{{STRING}}",
    "playerName": "{{STRING}}"
  }
}
```

Payload 参数说明

- token
 - 本 audio item 的 token
- offsetInMilliseconds
 - 当前的播放进度
- (optional)playerName
 - 播放器的类型，可选字段，如果不上报此字段，则默认当前事件上报来自音频播放器
 - SIMPLIFY_MODE 新增的 30s 听歌模式专用播放器

15. ClearQueue 指令

用于清除播放队列。ClearQueue 指令有两种行为（clearBehavior 参数），CLEAR_ENQUEUED 用于清除队列但保留当前正在播放的 audio item，CLEAR_ALL 用于清除播放队列并停止当前播放的 audio item。

消息样例

JSON

```
"directive": {
  "header": {
    "namespace": "ai.dueros.device_interface.audio_player",
```

```
    "name": "ClearQueue",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "clearBehavior": "{{STRING}}"
  }
}
```

Payload 参数说明

- clearBehavior
 - 清除行为，取值：CLEAR_ENQUEUED, CLEAR_ALL

16. PlaybackQueueCleared 事件

设备端处理完 ClearQueue 指令后上报此事件。

消息样例

```
JSON
"event": {
  "header": {
    "namespace": "ai.dueros.device_interface.audio_player",
    "name": "PlaybackQueueCleared",
    "messageId": "{{STRING}}"
  },
  "payload": {
  }
}
```

17. StreamMetadataExtracted 事件

设备端播放音频资源时，如果音频资源附带元信息，把元信息作为 key/value 对作为 JSON 对象进行上报本事件。元信息中的字符串和数字作为 JSON string，元信息中的 boolean 作为 JSON boolean，如果元信息中包含二进制数据（如图片、附件、应用程序数据），则不上传二进制部分的 key/value。

消息样例

```
JSON
```

```

"event": {
  "header": {
    "namespace": "ai.dueros.device_interface.audio_player",
    "name": "StreamMetadataExtracted",
    "messageId": "{{STRING}}"
  },
  "payload": {
    "token": "{{STRING}}",
    "metadata": {
      "{{STRING}}": "{{STRING}}",
      "{{STRING}}": {{BOOLEAN}}
      "{{STRING}}": "{{STRING NUMBER}}"
      ...
    }
  }
}

```

Payload 参数说明

- token
 - 本 audio item 的 token
- metadata
 - 元信息 key/value 对

18. Continue 指令

取消暂停/播放/继续

消息样例

```

JSON
"directive": {
  "header": {
    "namespace": "ai.dueros.device_interface.audio_player",
    "name": "Continue",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "token": "{{STRING}}"
  }
}

```

```
}
```

Payload 参数说明

- token
 - 下发指令时携带的 token

状态上报

消息样例

```
JSON
"clientContext": [
  {
    "header": {
      "namespace":
"ai.dueros.device_interface.audio_player",
      "name": "PlaybackState"
    },
    "payload": {
      "token": "{{STRING}}",
      "extension": "{{STRING}}",
      "offsetInMilliseconds": {{LONG}},
      "playerActivity": "{{STRING}}",
      "version" : "{{STRING}}",
      "playerName": "{{STRING}}"
    }
  }
]
```

Payload 参数说明

- (optional) token
 - 正在播放中的 audio item 的 token，当播放器没有播放资源时，可以不上报 token 字段
- (optional) extension
 - 扩展信息，Base64 格式字符串，存放的是 Play 指令中的 audioItem.extension
- (optional) offsetInMilliseconds

- 当前的播放进度，当播放器没有播放资源时，可以不上报 `offsetInMilliseconds` 字段
- `playerActivity`
 - 音频播放器当前状态，详见音频播放器摘要描述
 - 取值：PLAYING, STOPPED, PAUSED, BUFFER_UNDERRUN, FINISHED, IDLE
- (optional) `version`
 - 如果此字段存在，由 `audioplayer apk` 填充当前 `apk` 版本号
- (optional) `playerName`
 - 云端决策依赖端状态上报播放器的类型，字段缺失默认音乐播放器，场景电台播放器：SCENE_RADIO

播放器渲染指令

1. RenderPlayerInfo 指令

伴随 Play 指令输出，用于渲染音频播放器的主界面，RenderPlayerInfo 指令携带 audio 的内容元数据，用于支持设备端的界面渲染。

首先，token 参数用来关联 Play 指令，一次请求中服务端返回的 Play 和 RenderPlayerInfo 指令具有相同的 token。

其次，Play 指令和 RenderPlayerInfo 指令必须一起执行，例如，设备端上报 PlaybackNearlyFinished 事件后，收到服务端返回的 Play 和 RenderPlayerInfo 指令，这两个指令都需要加入本地队列，等待当前音频播放完毕后，取出来一起执行。

消息样例

```
JSON
{"directive": {
  "header": {
    "namespace":
      "ai.dueros.device_interface.screen_extended_card",
    "name": "RenderPlayerInfo",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "token": "{{STRING}}",
```



```
"onlyInBackground": "{{BOOLEAN}}",
"viewState": "{{ENUM}}",
"audioItemId": "{{STRING}}",
"linkFrom": "{{STRING}}",
"audioAlbumId": "{{STRING}}",
"content": {
  "audioItemType": "{{ENUM}}",
  "title": "{{STRING}}",
  "titleSubtext1": "{{STRING}}",
  "titleSubtext2": "{{STRING}}",
  "singerText": "{{STRING}}",
  "singerTextClickUrl": "{{STRING}}",
  "language": "{{STRING}}",
  "effect": {
    "bpm": {{INT}},
    "landscapeVideoUrl": "{{STRING}}",
    "portraitVideoUrl": "{{STRING}}",
    "fontColor": "{{STRING}}",
    "fontFamily": "{{STRING}}",
    "fontSize": "{{STRING}}",
    "landscapeBackgroundImageUrl": "{{STRING}}",
    "portraitBackgroundImageUrl": "{{STRING}}",
    "animation": {
      "id": "{{STRING}}",
      "url": "{{STRING}}",
      "md5": "{{STRING}}"
    }
  },
},
"ledEffect": {
  "theme": [{{STRING}}],
  "moodTag": "{{STRING}}",
  "beat": {{BeatStructure}}
},
"lyric": {{LyricStructure}},
"mediaLengthInMilliseconds": {{LONG}},
"art": {{ImageStructure}},
"isPaymentRequired": {{BOOLEAN}},
"linkClickedUrl": {{STRING}},
"favoriteReminder": {{STRING}},
"payment": {
  "membershipType": "{{ENUM}}",
  "type": "{{ENUM}}",
  "popup": {{BOOLEAN}},
  "icon": "{{STRING}},
```

```

        "title": "{{STRING}}",
        "url": "{{STRING}}",
        "appUrl": "{{STRING}}",
        "vipSquareUrl": "{{STRING}}",
        "images": [
            {{ImageStructure}},
            .....
        ],
        "isAudioCropped": {{BOOLEAN}}
    },
    "paymentInfo": {
        "hasPaid": {{BOOLEAN}},
        "description": "{{STRING}}",
        "payOptions": [
            {
                "name": "{{STRING}}",
                "buttonType": "{{ENUM}}",
                "link": {
                    "url": "{{STRING}}",
                    "params": {
                        "{{STRING}}": {{ANY}},
                        ...
                    }
                }
            }
        ]
    },
    "albumDetailUrl": "{{STRING}}",
    "customDetailUrl": {{STRING}},
    "customSplashScreenElements":
    {{CustomSplashScreenElementsStructure}},
    "provider": {
        "name": "{{STRING}}",
        "logo": {{ImageStructure}},
        "deepLink": "{{STRING}}",
        "appDownloadUrl": "{{STRING}}"
    },
    "mvUrl": "{{STRING}}",
    "playAudioUrl": "{{STRING}}",
    "playVideoUrl": "{{STRING}}",
    "promo": [
        {{PromoStructure}},
        .....
    ]

```

```

    "qualityMenu": [
        {{MenuStructure}},
    ],
    "isQualityPaymentRequired": {{BOOLEAN}},
    "equalizer": {
        "mode": "{{STRING}}",
        "url": "{{STRING}}",
        "refreshUrl": "{{STRING}}"
    },
    "albumCoverDisplayInMs": {{INTEGER}},
    "albumCoverAdvDisplayInMs": {{INTEGER}},
    "advertisement": {{AdvertisementRequestParamStructure}},
    "skinUrl": "{{STRING}}",
    "skinSwitchReminder": {{STRING}},
    "popupInfo": {
        "type": "{{STRING}}",
        "url": "{{STRING}}",
        "duration": "{{LONG}}",
        "images": [
            {{ImageStructure}},
            .....
        ],
        "appImages": [
            {{ImageStructure}},
            .....
        ],
        "popups": [
            {{PopupStructure}},
            .....
        ]
    },
    "animationVideoUrl": {{STRING}},
    "sceneTabName": {{STRING}},
    "sceneTabUrl": {{STRING}},
    "companion": {
        "limitType": {{ENUM}},
        "message": {{STRING}}
    }
},
"controls": [
    {{ControlStructure}},
    {{ControlStructure}},
    .....
],

```

```
"relatedResourceUrl":"{{STRING}}",
"subscript":{
  "iconUrl":"{{STRING}}",
  "text":"{{STRING}}",
},
"styleTemplate":"{{ENUM}}"
}
}
```

Payload 参数说明

- token
 - token, 和 Play 指令的 token 值一样
- onlyInBackground
 - 是否不显示播放界面, 只在后台 (下拉控制栏可见), 如果音频播放器全屏界面在显示时, 则关掉该界面。
 - 可选字段, 默认为 false
- (optional) viewState
 - 播放器需要展示的状态, 没有该字段时播放器维持当前展示状态, 取值参考端状态 ViewState 中 audioPlayer 的[viewState] (./screen.md)
- audioItemId
 - audio item 的 id
- (optional) linkFrom
 - audio 链接来源
- (optional) audioAlbumId
 - audio item 的专辑 id
- content
 - 关于音频内容的 key/value 属性对
- content.audioItemType
 - 音频的类型, 取值为:
 - MUSIC 音乐
 - SIMPLIFY_MODE 音乐 30s 听歌模式(sp >= 57 的 show 端支持此模式)下的音频
 - MUSIC_VIDEO 音乐 MV

- RADIO 直播电台，比如 AM 和 FM
- UNICAST 有声资源，比如相声等
- NEWS 新闻
- JOKE 笑话
- XIAODU_FM 小度电台
- CUSTOM_SPLASH_SCREEN 技能启动页
- BAIDU_PAN 百度网盘
- SCENE_RADIO 场景电台(sp >= 58show 版本支持此模式)下的音频
- content.title
 - 音频的标题，比如歌曲名，样例值：告白气球
- content.titleSubtext1
 - 音频子标题 1，比如歌手名，样例值：周杰伦-网易云音乐
- content.singerText
 - 音频的歌手名字文案，比如“周杰伦”。
- content.singerTextClickUrl
 - 音频 singerText，对应的 dueros linkClicked url
- content.titleSubtext2
 - 音频子标题 2，比如专辑名，样例值：周杰伦的床边故事
- content.mediaLengthInMilliseconds
 - 音频流的长度，单位为 ms，注意应该优先使用设备端拿到的音频流的实际长度，此字段更多是给 Companion App 等场景的渲染使用
- content.art
 - 音频封面图片或背景图片，比如音乐里专辑的封面图或者场景电台的背景图
- (optional) content.linkClickedUrl
 - 点播 url，用于音箱互联，当 query 某首歌之后，切换到音箱播放，然后把对应的 linkClickedUrl 通过 dlp 发送给指定的音箱设备
- (optional) content.isPaymentRequired
 - 是否要求此用户付费，取值为 true 表示需要此用户付费
 - 当歌曲为付费资源时，此字段的值不为空，
 - 如果用户已经付费，取值为 false，用户没有付费，取值为 true
- (optional) content.favoriteReminder

- 收藏红心上的提醒文案
- (optional) content.payment
 - 资源付费相关信息
- (optional) content.payment.type
 - 资源付费类型，取值为：
 - . NONE 免费资源
 - . PREMIUM 小度会员
 - . HIGH_QUALITY 会员高品质
 - . MUSIC_DIGITAL_ALBUM 数字专辑
 - . TME_UNION TME 联合会员
- (optional) content.payment.popup
 - 付费信息是否弹窗展示。取值为 true 时，端立即弹窗展示付费信息
- (optional) content.payment.membershipType
 - 会员身份类型，取值：
 - . NO_VIP 非小度会员
 - . XIAODU_VIP 小度黄金会员
 - . XIAODU_VIP_PLATINUM 小度白金会员
- (optional) content.payment.vipSquareUrl
 - 会员广场地址
- (optional) content.payment.icon
 - 付费说明的 icon url
- (optional) content.payment.title
 - 付费的文字说明
- (optional) content.payment.url
 - url 不需要渲染出来，需要跳转到会员落地页时，上报 LinkClicked 事件给服务端，事件参数里带上此 url
- (optional) content.payment.appUrl
 - appUrl 付费歌曲 app 点击跳转到支付页面的 url
- (optional) content.payment.images
 - 提示开通会员的图片
- (optional) content.payment.isAudioCropped

- 资源是否为试听片段（非完整的资源片段、仅试听 1 分钟）
- (optional) `content.paymentInfo`
 - 资源的购买信息
- `content.paymentInfo.hasPaid`
 - 是否已购买资源
- `content.paymentInfo.description`
 - 购买资源描述
- `content.paymentInfo.payOptions[]`
 - 资源购买按钮信息。可能会返回多个购买按钮，例如可以按专辑购买和按会员购买。
- `content.paymentInfo.payOptions[].name`
 - 购买按钮名称
- `content.paymentInfo.payOptions[].buttonType`
 - 购买类型，取值为：
 - `BUY_RESOURCE` 购买专辑
 - `BUY_THIRD_VIP` 购买第三方会员
- `content.paymentInfo.payOptions[].link`
 - 点击购买按钮的 `linkClicked` 事件参数
- `content.paymentInfo.payOptions[].link.url`
 - 点击购买按钮的 `linkClicked` URL
- (optional) `content.paymentInfo.payOptions[].link.params`
 - 点击购买按钮的 `linkClicked` params
- `content.albumDetailUrl`
 - 打开资源详情页的 `linkClicked` URL
- `content.provider`
 - 资源提供方信息
- `content.provider.name`
 - 资源提供方的名字，比如百度音乐、QQ 音乐、虾米音乐等
- `content.provider.logo`
 - 资源提供方的 logo
- (optional) `content.provider.deepLink`

- 调起资源提供方 App 的深度链接
- (optional) content.provider.appDownloadUrl
 - 下载资源提供方 App 的链接地址
- (optional) content.language
 - 语言，当前支持两种语言：zh(中文)，en(英文)。参考 [ISO 639-1](#)
- (optional) content.effect.bpm
 - 音乐每分钟节拍数，参考 [BPM](#)
- (optional) content.effect.landscapeVideoUrl
 - 播放器横屏模式特效背景视频 URL
- (optional) content.effect.portraitVideoUrl
 - 播放器竖屏模式特效背景视频 URL
- (optional) content.effect.fontColor
 - 播放器特效歌词字体颜色，格式为#RGB，如#AABBCC，端上默认颜色 #1A1A1A
- (optional) effect.fontFamily
 - 播放器特效歌词字体，如果该字段不存在或者值为空，则端上使用系统字体。当前可选值：
 - .duer_cang_er_yu_mo
 - .duer_qian_tu_ma_ke
 - .duer_qian_tu_xian_mo
 - .duer_rui_zhi_zhen_yan
 - .duer_zhuang_jia_ming_chao
- (optional) effect.fontSize
 - 播放器特效歌词字体大小，如果该字段不存在或值为空，则端上自行计算
- (optional) effect.landscapeBackgroundImageUrl
 - 播放器横屏模式特效背景图片 URL，当有背景动效时，通常为背景动效的首帧截图
- (optional) effect.portraitBackgroundImageUrl
 - 播放器竖屏模式特效背景图片 URL，当有背景动效时，通常为背景动效的首帧截图
- (optional) effect.animation.id
 - 当前逐句歌词 pag 动效模板集合 id

- (optional) `effect.animation.url`
 - 动效集合的压缩包地址
- (optional) `effect.animation.md5`
 - 动效集合压缩包的 md5 校验值（两个作用：1.校验压缩包下载是否完整，2.动效是否有更新）
- (optional) `content.ledEffect.moodTag`
 - 灯效情绪标签，当前可用于琉璃等设备可依据该字段识别情绪以渲染不同的音乐灯效
- (optional) `content.ledEffect.beat`
 - 灯效节拍数据，参考 `BeatStructure`
- (optional) `content.ledEffect.theme`
 - 灯效主题数据，用于在默认（智能）场景下设备端渲染音乐灯效
- (optional) `content.lyric`
 - 歌词的结构，参考 `LyricStructure`
- (optional) `content.customDetailUrl`
 - 音频的文本详情地址,比如 tts 新闻的链接 url
- (optional) `content.customSplashScreenElements`
 - 技能启动页的结构，参考 `CustomSplashScreenElementsStructure`，此字段是给支持技能启动页本地渲染使用，只有 `content.audioItemType` 为 `CUSTOM_SPLASH_SCREEN` 时，才会有此字段
- (optional) `content.mvUrl`
 - 资源对应 mv 播放链接(app 上播放 MV 资源使用过)
- (optional) `content.playAudioUrl`
 - 资源音频播放的 `LinkClicked Url`
- (optional) `content.playVideoUrl`
 - 资源视频播放的 `LinkClicked Url`
- (optional) `content.promo`
 - 促销推广位，结构参考 `PromoStructure`
- (optional) `content.qualityMenu[]`
 - 资源可选择的品质, 高品质/标准品质, `MenuStructure` 结构
- (optional) `content.isQualityPaymentRequired`
 - 其他品质资源是否需要付费。取值为 `true` 且用户点击音频品质时，不出品质

选择列表，直接弹窗让用户购买会员。

- (optional) content.equalizer
 - 资源的音效信息
- (optional) content.equalizer.mode
 - 正在使用的音效名称
- (optional) content.equalizer.url
 - 点击音效跳转到音效选择页面的 url
- (optional) content.equalizer.refreshUrl
 - 更新音效信息的 url。当端上因恢复出厂设置等原因导致音效数据丢失时，调用 refreshUrl 重新设置音效数据。
- (optional) content.albumCoverDisplayInMs
 - 专辑封面展示时长(单位为毫秒)
- (optional) content.albumCoverAdvDisplayInMs
 - 专辑封面被替换成广告后，广告要展示多长时间(单位为毫秒)
- (optional)content.advertisement
 - 播放器请求云端获取广告的参数，参考 AdvertisementRequestParamStructure
- (optional) content.skinUrl
 - 播放器主题皮肤中心地址
- (optional) content.skinSwitchReminder
 - 播放器一键切换皮肤提示
- (optional) content.popupInfo
 - 播放器弹窗信息
- (optional) content.popupInfo.type
 - 弹窗类别，取值为：
 - . PREMIUM 小度会员
 - . HIGH_QUALITY 会员高品质
 - . PENNY 1 分钱会员活动
 - . TME_UNION TME 联合会员
- (optional) content.popupInfo.url
 - 弹窗的点击 url

- (optional) content.popupInfo.duration
 - 弹窗持续的时间
- (optional) content.popupInfo.images
 - show 端弹窗的图片， ImageStructure 结构(Show 端 SP59 及以下使用)
 - 图片类型：基本的静态图片，比如 jpg,png,jpeg
 - 图片大小：宽： 710dp 高： 416dp
- (optional) content.popupInfo.applImages
 - app 弹窗的图片， ImageStructure 结构
 - 图片类型：基本的静态图片，比如 jpg,png,jpeg
 - 图片大小：宽： 710dp 高： 416dp
- (optional) content.popupInfo.popups
 - show 端弹窗的展示信息， PopupStructure 结构(Show 端 SP60 及以上使用)
- (optional) content.animationVideoUrl
 - 场景电台动效视频链接
- (optional) content.sceneTabName
 - (场景电台适用) 当前电台所属 tab 的名称
- (optional) content.sceneTabUrl
 - (场景电台适用) 当前电台所属 tab 资源刷新 url
- (optional) content.companion
 - 陪伴模式下提示信息
- (optional) content.companion.limitType
 - 陪伴模式下播放受限类型
 - PREMIUM: 当前用户受到会员阻断
 - PREMIUM_COMPANY: 结对用户受到会员阻断
 - PREMIUM_BOTH: 双方受到会员阻断
- (optional) content.companion.message
 - 陪伴模式下无版权、会员阻断等提示文案
- controls[]
 - 控件列表
- (optional) relatedResourceUrl

- 获取播放资源相关推荐信息的链接地址, 上报 LinkClick 事件给服务端
- (optional) subscript
 - 图片下标信息, 若有下标 iconUrl、text 项至少有一个, 无下标时可以不输出
- (optional) subscript.iconUrl
 - 图片下标图标链接地址
- (optional) subscript.text
- (optional) styleTemplate
 - 播放信息展示类型(注: 只供 IOV 车联网使用, 默认不下发此字段意义与 IOV_FULL_SCREEN_STYLE 一致), 取值为:
 - .IOV_FULL_SCREEN_STYLE 全部展示
 - .IOV_PLAYER_BAR_STYLE 部分展示

音频播放器支持的控件列表

控件名	类型	功能	选项值和意义
PLAY_PAUSE	BUTTON	暂停/继续	
PREVIOUS	BUTTON	上一首	
NEXT	BUTTON	下一首	
LYRIC	BUTTON	显示/隐藏歌词	
REPEAT	RADIO_BUTTON	循环模式切换	REPEAT_ONE:单曲 REPEAT_ALL:列表 SHUFFLE: 随机播
FAVORITE	BUTTON	收藏/取消	
DEVICE_FAVORITE	BUTTON	收藏/取消(为新增与 FAVORITE互斥, 优先使用	

点击图片可查看完整电子表格

1.1 LyricStructure 结构

定义歌词的结构。

消息样例

```
JSON
{
  "url": "{{STRING}}",
  "format": {{ENUM}}
```

```
}
```

参数说明

- url
 - 歌词 url, url 可以为 http 协议; 也可以支持 cid 格式, 即内容直接附加在指令中
- format
 - 歌词的格式, 支持的格式类型:
 - .LRC: 参考
 - .DRC: 逐字/词歌词

1.2 BeatStructure 结构

定义节奏数据的结构

消息样例

```
JSON
{
  "url": "{{STRING}}",
  "format": {{ENUM}}
}
```

参数说明

- url
 - 节奏数据的 url, 通常为 http 协议地址
- format
 - 数据格式

1.3 PromoStructure 结构

定义推广促销位的结构。

消息样例

```
JSON
{
```

```
"type": {{ENUM}},
"image": {{ImageStructure}},
"closeIcon": {{ImageStructure}},
"closeUrl": "{{STRING}}",
"timeoutInMilliseconds": {{LONG}},
"url": "{{STRING}}"
}
```

参数说明

- type
 - 推广位类型，支持的格式类型：
 - TOP_PROMO: 在顶部位置展示
 - LEFT_TOP_PROMO: 在左上角位置展示
 - RIGHT_BOTTOM_PROMO: 在右下角位置展示
 - RIGHT_TOP_PROMO: 在右上角位置展示
- image
 - 推广位展示的图片
- closeIcon
 - 关闭按钮的 icon
- closeUrl
 - 点击关闭时上报的事件地址，目的是让云端知道是用户手动关闭，为下发策略提供支持
- timeoutInMilliseconds
 - 显示的时长（秒），为 0 时一直不主动消失
- url
 - 点击的事件地址

1.4 CustomSplashScreenElementsStructure 结构

定义技能启动页的结构。

消息样例

```
JSON
{
```

```
"backgroundImageUrl": {{ImageStructure}},
"logo": {{ImageStructure}},
"title": "{{STRING}}",
"introduction": "{{STRING}}",
"characterImage": {{ImageStructure}}
}
```

参数说明

- (optional) backgroundImageUrl
 - 技能启动页背景图
- (optional) logo
 - 技能 logo 图
- (optional) title
 - 技能欢迎语的标题
- (optional) introduction
 - 技能介绍
- (optional) characterImage
 - 技能人物形象图

1.5 MenuStructure 结构

```
JSON
{
  "name": "{{STRING}}",
  "clickUrl": "{{STRING}}",
  "active": {{BOOLEAN}},
  "icon": {{ImageStructure}},
  "image": {{ImageStructure}},
  "appImage": {{ImageStructure}},
  "appMenuIcon": "{{STRING}}"
}
```

参数说明

- name
 - 菜单项名称/标题
- clickUrl

- 点击选中时调用 URL
- active
 - 菜单项是否被选中，true 表示被选中，false 表示未被选中
- (optional) icon
 - 菜单项对应的 icon 图片
- (optional) image
 - 非会员切换高品质弹窗信息，ImageStructure 结构
 - 图片类型：基本的静态图片，比如 jpg,png,jpeg
 - 图片大小：宽：710dp 高：416dp
- (optional) appImage
 - 非会员切换高品质 app 弹窗信息，ImageStructure 结构
- (optional) appMenuIcon
 - 品质对应的图标，用于在 app 上展示

天气垂类指令

1. RenderWeather 指令

查询某地天气

- 例如：query 是北京天气怎么样、北京明天天气怎么样

消息样例

```
JSON
{
  "directive": {
    "header": {
      "namespace":
        "ai.dueros.device_interface.screen_extended_card",
      "name": "RenderWeather",
      "messageId": "{{STRING}}",
      "dialogRequestId": "{{STRING}}"
    },
    "payload": {
      "token": "{{STRING}}",
      "country": "{{STRING}}",
      "province": "{{STRING}}",

```



```
"city": "{{STRING}}",
"county": "{{STRING}}",
"hasCounty": "{{BOOLEAN}}",
"timeZone": "{{STRING}}",
"description": "{{STRING}}",
"askingDay": "{{STRING}}",
"askingDate": "{{STRING}}",
"askingDateDescription": "{{STRING}}",
"askingPeriod": "{{STRING}}",
"weatherPeriodRange": "{{ENUM}}",
"askingType": "{{STRING}}",
"weatherForecast": [
  {
    "currentTemperature": "{{STRING}}",
    "currentPM25": {{LONG}},
    "currentAirQuality": "{{STRING}}",
    "pm25": {{LONG}},
    "airQuality": "{{STRING}}",
    "displayAirQuality": "{{STRING}}",
    "weatherIcon": "{{ImageStructure}}",
    "highTemperature": "{{STRING}}",
    "lowTemperature": "{{STRING}}",
    "day": "{{STRING}}",
    "date": "{{STRING}}",
    "weatherCondition": "{{STRING}}",
    "displayWeatherCondition": "{{STRING}}",
    "windCondition": "{{STRING}}",
    "indexes": [
      {
        "type": "{{STRING}}",
        "level": "{{STRING}}",
        "suggestion": "{{STRING}}"
      },
      ...
    ]
  },
  ...
],
"hoursInfo": [
  {
    "icon": "{{STRING}}"
    "weather": "{{STRING}}",
    "hour": "{{STRING}}",
    "temperature": "{{STRING}}",
```

```

        "pm25": "{{STRING}}",
        "windPower": "{{STRING}}"
    },
    ...
],
"alarms": [
    {
        "type": "{{STRING}}",
        "city": "{{STRING}}",
        "content": "{{STRING}}",
        "level": "{{STRING}}",
        "displayLevel": "{{STRING}}",
        "publishDatetime": "{{STRING}}"
    },
    ...
]
}
}

```

Payload 参数说明

- token
 - 本页面的唯一标识
- (optional) country
 - 国家
- (optional) province
 - 省份
- city
 - 城市
- county
 - 县/区
- (optional) hasCounty
 - 由于地理位置目前是精确到区县一级，而天气信息有时候只精确到城市级别，协议之前定义区县 county 字段为非可选字段，为了保持播报与显示的一致性，增加 hasCounty 字段，指导设备显示精确到城市还是区县。
 - true 设备显示精确到区县
 - false 设备显示精确到城市

- **timeZone**
 - 时区，用户询问地区的时区
 - 比如用户问"火奴鲁鲁天气"，**timeZone** 是火奴鲁鲁的时区，值为"-10"，表示西十区
 - 比如用户问"北京天气"，则 **timeZone** 是北京的时区，值为"8"，表示东八区
- **description**
 - 回答用户问题的描述，比如用户 **query** 为"今天下雨吗"，此样例值如下："北京今天没有雨，多云，-4℃ ~ 7℃"
- **askingDay**
 - 周几，值如下：
 - "MON": 周一
 - "TUE": 周二
 - "WED": 周三
 - "THU": 周四
 - "FRI": 周五
 - "SAT": 周六
 - "SUN": 周日
- **askingDate**
 - 日期，ISO 8601 格式
 - "yyyy-mm-dd"
- (optional) **askingDateDescription**
 - 日期的描述信息，样例值："今天"、"明天"、"后天"、"大后天"等
- (optional) **askingPeriod**
 - 描述未来多日日期的闭区间，由两个单日日期组成，"yyyy-mm-dd,yyyy-mm-dd"，第一个是起始日期，第二个是结束日期，其中单日日期是 ISO 8601 格式
 - **askingPeriod** 和 **askingDate** 相比，优先级更高，即当两者同时存在时，表明用户询问的是多日天气，此时 **askingPeriod** 更能准确描述用户询问日期
 - 由于向后兼容的缘故，当问多日天气时，**askingDate** 依然保持原来的值，即设置为起始单日日期
 - 如今天时间为 2017 年 11 月 24 日，用户询问 **query**"未来三天天气"，**askingPeriod** 样例值如下："2017-11-25,2017-11-27"，**askingDate** 样例值如

下:"2017-11-25"

. 如今天时间为 2017 年 11 月 24 日, 用户询问 query"未来一周天气", askingPeriod 样例值如下: "2017-11-25,2017-12-01", askingDate 样例值如下:"2017-11-25"

. 如今天时间为 2017 年 11 月 24 日, 星期五, 用户询问 query"下周北京天气", askingPeriod 样例值如下: "2017-11-27,2017-12-03", askingDate 样例值如下:"2017-11-27"

- (optional) weatherPeriodRange

- 荣耀定制需求, 根据 **【askingPeriod】** 字段判断, 当前请求返回的天气结果是日卡还是周卡, 如果 **【askingPeriod】** 跨天, 返回周卡, 反之则返回日卡, 取值如下:

- . "daily": 日卡

- . "weekly": 周卡

- askingType

- 天气类别, 枚举值如下:

- . "WEATHER":天气类别, query 示例: 今天天气

- . "RAIN":下雨类别, query 示例: 今天下雨吗

- . "SNOW":下雪类别, query 示例: 今天下雪吗

- . "FOG":雾的类别, query 示例: 今天有雾吗

- . "WIND":风的类别, query 示例: 今天有风吗

- . "CLOUDY":云的类别, query 示例: 今天多云吗

- . "SUNNY":晴天的类别, query 示例: 今天是晴天吗

- . "AQI":空气质量类别, query 示例: 今天有雾霾吗

- . "TEMPERATURE":温度类别, query 示例: 今天温度

- . "HIGH_TEMPERATURE":高温类别, query 示例: 今天热吗

- . "LOW_TEMPERATURE":低温类别, query 示例: 今天冷吗

- . "CLOTHES":穿衣指数类别, query 示例: 今天穿衣指数

- . "EXERCISE":运动指数类别, query 示例: 今天适合运动吗

- . "INFLUENZA":感冒指数类别, query 示例: 今天容易感冒吗

- . "TRIP":旅游指数类别, query 示例: 今天适合爬山吗

- . "ULTRAVIOLET":紫外线指数类别, query 示例: 今天紫外线强吗

- "CAR_WASH":洗车指数类别, query 示例: 今天适合洗车吗
 - "SUN_RISE_SET":日出日落类别, query 示例: 今天什么时候日出
 - "RAIN_HOUR":小时级下雨类别, query 示例: 今天几点有雨
- weatherForecast[]
 - 未来几天的天气, 数组里第一个数据并非一定是今天天气数据, 数组个数由于后续扩展会变化, 请按日期取相应数据
- weatherForecast[].weatherIcon
 - 天气的图标 url
 - 见下文 ImageStructure 结构
- (optional) weatherForecast[].currentTemperature
 - 当前温度, 有两种格式的返回结果, 当结果是今天天气时, 有该字段
 - "14℃": 摄氏温度
 - "55°F": 华氏温度
- (optional) weatherForecast[].currentPM25
 - 当本 weatherForecast 项中的 date 与今天的日期相等时, 有该字段, PM2.5 当前值, 整数
- (optional) weatherForecast[].currentAirQuality
 - 当本 weatherForecast 项中的 date 与今天的日期相等时, 有该字段, 当前空气质量, 样例值: "优"、"良好"、"严重"等
- (optional) weatherForecast[].pm25
 - 由于扩展到每天都可能有 PM2.5 的值, 于是增加该字段, 当本 weatherForecast 项中的 date 与今天的日期相等时, 则会同时存在 currentPM25 和 pm25, 且二者相等, 否则, 若有该字段, 代表本 weatherForecast 项中对应日期的 PM2.5 的值
- (optional) weatherForecast[].airQuality
 - 由于扩展到每天都可能有 PM2.5 的值, 于是增加该字段, 当本 weatherForecast 项中的 date 与今天的日期相等时, 则会同时存在 currentAirQuality 和 airQuality, 且二者相等, 否则, 若有该字段, 代表本 weatherForecast 项中对应日期的空气质量, 样例值: "优"、"良好"、"严重"等
 - 最高温度, 有两种格式的返回结果
 - "14℃": 摄氏温度
 - "55°F": 华氏温度

- (optional) `weatherForecast[].displayAirQuality`
 - 含义同 `airQuality`；区别是：音箱端支持多语种能力时，该字段需要被翻译，用于界面展示
- `weatherForecast[].lowTemperature`
 - 最低温度，有两种格式的返回结果
 - "14°C": 摄氏温度
 - "55°F": 华氏温度
- `weatherForecast[].day`
 - 周几，值如下：
 - "MON": 周一
 - "TUE": 周二
 - "WED": 周三
 - "THU": 周四
 - "FRI": 周五
 - "SAT": 周六
 - "SUN": 周日
- `weatherForecast[].date`
 - 日期，ISO 8601 格式
 - "yyyy-mm-dd"
- `weatherForecast[].weatherCondition`
 - 天气情况，样例值："多云转阴"、"晴"等
- (optional) `weatherForecast[].displayWeatherCondition`
 - 含义同 `weatherCondition`；区别是：音箱端支持多语种能力时，该字段需要被翻译，用于界面展示
- `weatherForecast[].windCondition`
 - 风力情况，样例值："南风微风"、"微风"等
- (optional) `weatherForecast[].indexes[]`
 - 每一天的各种指数，比如穿衣指数，洗车指数
 - 目前只有当日的天气数据，才会有此指数，也就是 `weatherForecast[0]`才会有此指数
- `weatherForecast[].indexes[].type`

- 指数的名称,目前当 `date` 是今天日期, 有指数 6 种, 其余 `date` 没有 `indexes` 字段, 枚举值如下:
 - . "CLOTHES": 穿衣指数
 - . "EXERCISE": 运动指数
 - . "INFLUENZA": 感冒指数
 - . "TRIP": 旅游指数
 - . "ULTRAVIOLET": 紫外线指数
 - . "CAR_WASH": 洗车指数
- `weatherForecast[].indexes[].level`
 - 指数的等级, 比如对于穿衣指数, 样例值是"热", "很热"
- `weatherForecast[].indexes[].suggestion`
 - 指数对应的建议话术, 比如对于穿衣指数, 若在 `level` 是热的情况下, 样例值是"天气热, 建议着短裙、短裤、短薄外套、T 恤等夏季服装"
- (optional) `hoursInfo[]`
 - 小时级天气信息 只返回排好序的未来 12 小时的数据
- `hoursInfo[].hour`
 - 小时具体时间 样例值: 2021092414
- `hoursInfo[].icon`
 - 小时级天气图标
- `hoursInfo[].weather`
 - 小时级天气气象
- `hoursInfo[].temperature`
 - 小时级气温
- `hoursInfo[].pm25`
 - 小时级空气质量指数
- `hoursInfo[].windPower`
 - 小时级风力
- (optional) `alarms[]`
 - 预警信息,可能会有多个预警, 此字段只有少数请求才会有
- `alarms[].type`
 - 预警类型, 样例值: "雷电", "高温"

- `alarms[].city`
 - 预警城市，样例值: "延边"
- `alarms[].content`
 - 预警原始内容，样例值: "预计未来 12 小时，我州大部分地方有雷电活动，局部地方伴有强降水..."
- `alarms[].level`
 - 预警等级，样例值: "蓝色", "黄色", "橙色", "红色"
- (optional) `alarms[].displayLevel`
 - 含义同 `level`；区别是：音箱端支持多语种能力时，该字段需要被翻译，用于界面展示
- `alarms[].publishDatetime`
 - 预警发布时间，ISO 8601 格式
 · "yyyy-mm-ddThh:mm:ss"，样例值: "2017-12-11T14:30:08"

2. RenderAirQuality 指令

查询某地空气质量

- 例如: `query` 是北京空气怎么样

消息样例

```
JSON
{
  "directive": {
    "header": {
      "namespace":
        "ai.dueros.device_interface.screen_extended_card",
      "name": "RenderAirQuality",
      "messageId": "{{STRING}}",
      "dialogRequestId": "{{STRING}}"
    },
    "payload": {
      "token": "{{STRING}}",
      "city": "{{STRING}}",
      "pm25": {{LONG}},
      "airQuality": "{{STRING}}",
      "day": "{{STRING}}",
      "date": "{{STRING}}",
      "dateDescription": "{{STRING}}",
```



```
"tips":"{{STRING}}"  
}  
}
```

Payload 参数说明

- token
 - 本页面的唯一标识
- city
 - 城市
- pm25
 - PM2.5 当前值，整数
- airQuality
 - 空气质量，样例值："优"、"良好"、"严重"等
- day
 - 当前是周几，可能的值如下
 - "MON": 周一
 - "TUE": 周二
 - "WED": 周三
 - "THU": 周四
 - "FRI": 周五
 - "SAT": 周六
 - "SUN": 周日
- date
 - 当前日期，ISO 8601 格式
 - "yyyy-mm-dd"
- dateDescription
 - (optional)日期的描述信息，样例值："今天"、"明天"、"后天"、"大后天"等
- tips
 - 提示，样例值："可以正常在户外活动，易敏感人群应减少外出。"

信息百科相关指令

1. RenderStock 指令

查询股票信息

- 例如：query 是百度今天股价怎么样

消息样例

JSON

```
"directive": {
  "header": {
    "namespace":
"ai.dueros.device_interface.screen_extended_card",
    "name": "RenderStock",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "token": "{{STRING}}",
    "changeInPrice": {{DOUBLE}},
    "changeInPercentage": {{DOUBLE}},
    "code": "{{STRING}}",
    "marketPrice": {{DOUBLE}},
    "marketStatus": "{{STRING}}",
    "marketName": "{{STRING}}",
    "name": "{{STRING}}",
    "marketPriceDatetime": "{{STRING}}",
    "marketTimezoneDescription": "{{STRING}}",
    "openPrice": {{DOUBLE}},
    "previousClosePrice": {{DOUBLE}},
    "dayHighPrice": {{DOUBLE}},
    "dayLowPrice": {{DOUBLE}},
    "priceEarningRatio": {{DOUBLE}},
    "marketCap": {{LONG}},
    "dayVolume": {{LONG}}
  }
}
```

Payload 参数说明

- token
 - 本页面的唯一标识
- changeInPrice
 - 股价相对前一天收盘价的价格变化，样例值：5.47，-5.1

- **changeInPercentage**
 - 股价相对前一天收盘价的价格变化率，样例值：0.021, -0.003
- **marketPrice**
 - 股价，样例值：189, 10.12，价格单位是股票所在交易所的货币，如美股是美元，港股是港元，A股是人民币
- **code**
 - 股票代码，样例值：BIDU, 000001, 00700
- **marketStatus**
 - 交易所营业情况，样例值："已收盘"、"交易中"、"休市"
- **marketName**
 - 样例值："沪深"、"美股"、"港股"、"指数"
- **name**
 - 股票的中文名称，样例值："谷歌"、"百度"、"万科"
- **marketPriceDatetime**
 - 格式为 ISO 8601，如果股市处于交易中，时间为当前时间，如果股市已经休市，时间为休市时的时间
 - "yyyy-mm-ddThh:mm:ss"
- **marketTimezoneDescription**
 - 样例值："北京时间"、"美东时间"，沪深股市和港股的时间是北京时间，美股的时间是美东时间
- **openPrice**
 - 开盘价，样例值：189, 10.12，价格单位是股票所在交易所的货币，如美股是美元，港股是港元，A股是人民币
- **previousClosePrice**
 - 上一天交易日的收盘价，样例值：189, 10.12 等，价格单位是股票所在交易所的货币，如美股是美元，港股是港元，A股是人民币
- **dayHighPrice**
 - 当天截至查询时的最高价，样例值：189, 10.12 等，价格单位是股票所在交易所的货币，如美股是美元，港股是港元，A股是人民币
- **dayLowPrice**
 - 当天截至查询时的最低价，样例值：189, 10.12 等，价格单位是股票所在交易所的货币，如美股是美元，港股是港元，A股是人民币

- priceEarningRatio
 - 市盈率，样例值：20，19.12 等
- marketCap
 - 市值，整数，样例值：12011200000，6450200000，单位是股票所在交易所的货币，如美股是美元，港股是港元，A 股是人民币
- dayVolume
 - 成交量，股票的股份成交数量，整数，样例值：100000

2. RenderBaike 指令

查询百科信息

- 例如：介绍一下刘德华

消息样例

```
JSON
{
  "directive": {
    "header": {
      "namespace":
        "ai.dueros.device_interface.screen_extended_card",
      "name": "RenderBaike",
      "messageId": "{{STRING}}",
      "dialogRequestId": "{{STRING}}"
    },
    "payload": {
      "token": "{{STRING}}",
      "title": "{{STRING}}",
      "subTitle": "{{STRING}}",
      "introduction": "{{STRING}}",
      "image": "{{ImageStructure}}",
      "mediaResources": [
        {
          "audioUrl": "{{STRING}}",
          "sourceType": "{{ENUM}}",
          "videoUrl": "{{STRING}}"
        },
        ...
      ],
      "link": {{LinkStructure}}
    }
  }
}
```

- token
 - 本页面的唯一标识
- title
 - 百科的标题， 样例值："李白"、"朱元璋"、"周杰伦"
- (optional) subTitle
 - 百科的副标题， 样例值："唐朝著名浪漫主义诗人"
- introduction
 - 百科介绍的详细内容
- (optional) image
 - 百科结果如果有配图， 则该字段提供图片数据
- (optional) mediaResources[]
 - 多媒体资源数据， 数组结构， 主要存放音频、来源、视频这些数据
- (optional) mediaResources[].audioUrl
 - 百科的音频数据资源链接
- (optional) mediaResources[].sourceType
 - 多媒体资源的来源
 - . MIAODONG_BAIKE 来自秒懂百科
 - . DEFAULT 其他来源
- (optional) mediaResources[].videoUrl
 - 百科的视频数据资源链接
- link
 - 百科的跳转落地页链接

扬声器控制

摘要

本接口定义扬声器控制相关的功能，如音量的调节、静音设置等等。

1. SetVolume 指令

设置音量，给绝对音量值，音量绝对值在 0 到 100 之间。

消息样例

```
json
{
  "directive": {
    "header": {
      "namespace":
        "ai.dueros.device_interface.speaker_controller",
      "name": "SetVolume",
      "messageId": "{{STRING}}",
      "dialogRequestId": "{{STRING}}"
    },
    "payload": {
      "volume": {{LONG}}
    }
  }
}
```

Payload 参数说明

- volume
 - 要设置的音量绝对值，[0, 100]

2. AdjustVolume 指令

设置音量，给相对变化值。

消息样例

```
json
{
  "directive": {
    "header": {
      "namespace":
        "ai.dueros.device_interface.speaker_controller",
      "name": "AdjustVolume",
      "messageId": "{{STRING}}",
      "dialogRequestId": "{{STRING}}"
    },
    "payload": {
```

```
        "volume": {{LONG}}
    }
}
```

Payload 参数说明

- volume
 - 音量调整相对值, [-100, 100]

3. SetMute 指令

静音/取消静音。

消息样例

```
sql
"directive": {
  "header": {
    "namespace":
"ai.dueros.device_interface.speaker_controller",
    "name": "SetMute",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "mute": {{BOOLEAN}}
  }
}
```

Payload 参数说明

- mute
 - 静音设置: true 为设置静音; false 为取消静音

屏幕亮度控制

1. SetBrightness 指令

设置亮度, 给绝对亮度值, 亮度绝对值在 0 到 100 之间。

消息样例

```
json
{
  "directive": {
    "header": {
      "namespace":
        "ai.dueros.device_interface.display_controller",
      "name": "SetBrightness",
      "messageId": "{{STRING}}",
      "dialogRequestId": "{{STRING}}"
    },
    "payload": {
      "brightness": {{LONG}}
    }
  }
}
```

Payload 参数说明

- brightness
 - 要设置的亮度绝对值，[0, 100]

2. AdjustBrightness 指令

设置亮度，给相对变化值。

消息样例

```
json
{
  "directive": {
    "header": {
      "namespace":
        "ai.dueros.device_interface.display_controller",
      "name": "AdjustBrightness",
      "messageId": "{{STRING}}",
      "dialogRequestId": "{{STRING}}"
    },
    "payload": {
      "brightness": {{LONG}}
    }
  }
}
```


Payload 参数说明

- brightness
 - 亮度调整相对值, [-100, 100]

设备硬件控制

1. 蓝牙

打开或关闭蓝牙功能

消息样例

```
JSON
{
  "header": {
    "namespace":
    "ai.dueros.device_interface.extensions.device_control",
    "name": "SetBluetooth",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "bluetooth": {{BOOLEAN}}
  }
}
```

Payload 参数说明

- bluetooth
 - true: 打开, false: 关闭

2. 打开应用/功能

打开端上应用或功能

消息样例

```

json
{
  "header": {
    "namespace":
    "ai.dueros.device_interface.extensions.device_control",
    "name": "Open",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "app_name": "{{STRING}}"
  }
}

```

Payload 参数说明

- app_name
 - 应用或功能名称（eg：自动亮度、省电模式、相机、相册、计算器、微信、短信）

3. 关闭应用/功能

关闭端上应用或功能

消息样例

```

json
{
  "header": {
    "namespace":
    "ai.dueros.device_interface.extensions.device_control",
    "name": "Close",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "app_name": "{{STRING}}"
  }
}

```

Payload 参数说明

- app_name
 - 应用或功能名称（eg：自动亮度、省电模式、相机、相册、计算器、微信、短信、收音机）

4. 屏幕显示

打开/关闭屏幕显示

消息样例

```
json
{
  "header": {
    "namespace":
    "ai.dueros.device_interface.extensions.device_control",
    "name": "Display",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "action": "{{ENUM}}"
  }
}
```

Payload 参数说明

- action
 - On: 打开, Off: 关闭

5. 打开媒体播放

所有参数为可选参数, 有可能 payload 为空, 代表泛意图, 如: "播放歌曲"

消息样例

```
json
{
  "header": {
    "namespace":
    "ai.dueros.device_interface.extensions.device_control",
```

```
    "name": "MusicPlay",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "action": "{{STRING}}",
    "artist": "{{STRING}}",
    "song": "{{STRING}}",
    "tag": "{{STRING}}",
    "genre": "{{STRING}}",
    "sortType": "{{STRING}}",
    "language": "{{STRING}}"
  }
}
```

Payload 参数说明

- action
 - PLAY: 播、放、听
 - SEARCH: 搜、查
 - SING: 唱
- artist
 - 歌手/作词/作曲, 多个艺术家使用英文逗号分隔, e.g. 张惠妹,张韶涵
- song
 - 歌名
- tag
 - 资源标签, eg. 浪漫的歌曲, 填写: 浪漫的, 多个 tag 使用英文逗号分隔, e.g. 浪漫,校园
- genre
 - 歌曲流派, 区别于 tag
 - Jazz: 爵士
 - Rock: 摇滚
 - Classical: 古典
 - Hip-Hop: 嘻哈
 - Electronic: 电子
 - Blues: 布鲁斯, 蓝调

- Reggae: 雷鬼
- Country: 乡村
- Pop: 流行
- Folk: 民谣
- RAP: 说唱
- Ancientry: 古风
- R&B: 节奏布鲁斯
- HeavyMetal: 重金属
- NewWorld: 新世纪
- Punk: 朋克
- sortType
 - 搜索排序规则，多个以英文逗号分隔，类似最近很火的英文个，SortType=NEW,HOT
 - HIGH_SCORE, 高分，好评
 - PLAY_COUNT, eg. 播放 QQ 音乐里播放量第一的歌曲
 - NEW, eg. 播放最新的歌曲
 - HOT, eg. 最热的歌曲
- language
 - 语种: 如: 国语, 英语, 法语, 粤语, 韩语, 日语 等等

6. 媒体播放控制

消息样例

```
json
{
  "header": {
    "namespace":
    "ai.dueros.device_interface.extensions.device_control",
    "name": "MediaControl",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
```

```
    "action": "{{STRING}}",
    "index": "{{STRING}}",
    "mode": "{{ENUM}}",
    "orientation": "{{ENUM}}",
    "time": "{{STRING}}"
  }
}
```

Payload 参数说明

- action
 - Pause: 暂停, 停止播放
 - Exit: 关闭音乐
 - Continue: 继续播放
 - Previous: 上一首
 - Next: 下一首
 - SelectIndex: 播放第几个
 - PlayMode: 播放模式
 - ADJPlayTime 快进/快退
 - AddFavorites 歌曲收藏
- index (optional)
 - 播放第几个, 从 1 开始; 当 action=SelectIndex 时下发
- mode (optional)
 - REPEAT: 单曲循环, SEQUENCE: 顺序播放; 当 action=PlayMode 时下发
- orientation (optional)
 - 播放方向, FORWARD:快进、REWIND: 快退; 当 action=ADJPlayTime 时下发
- time (optional)
 - 快进/快退的时间, eg: 15 秒; 当 action=ADJPlayTime 时下发

闹钟控制

1. SetAlert 指令

设备端在收到 **SetAlert** 指令时，应该在本地设置一个闹钟，到设定的时间能够自动触发闹铃。在用户通过语音请求设置闹钟时，或者通过 **Companion App** 重新启用先前设置的闹钟时，都有可能使服务端下发该指令。

消息样例

```
json
{
  "directive": {
    "header": {
      "namespace": "ai.dueros.device_interface.alerts",
      "name": "SetAlert",
      "messageId": "{{STRING}}",
      "dialogRequestId": "{{STRING}}"
    },
    "payload": {
      "token": "{{STRING}}",
      "type": "{{STRING}}",
      "scheduledTime": "{{STRING}}"
    }
  }
}
```

Payload 参数说明

- token
 - 本闹钟的唯一 token
- type
 - 闹钟类型
 - 取值
 - "TIMER": 定时器
 - "ALARM": 闹钟
- scheduledTime
 - 触发时间，ISO 8601 格式, 示例: 2025-03-08T00:00:02+0000

电话短信

本接口提供了打电话和发短信相关的指令，通过本接口定义的指令可以控制设备端给

指定联系人打电话和发短信

1. PhonecallByName 指令

向通讯录的联系人拨打电话

消息样例

```
json
{
  "directive": {
    "header": {
      "namespace":
        "ai.dueros.device_interface.extensions.telephone",
      "name": "PhonecallByName",
      "messageId": "{{STRING}}",
      "dialogRequestId": "{{STRING}}"
    },
    "payload": {
      "name": "{{STRING}}"
    }
  }
}
```

Payload 参数说明

- name
 - 联系人名称

2. PhonecallByNumber 指令

向唯一的电话号码拨打电话

消息样例

```
json
{
  "directive": {
    "header": {
      "namespace":
        "ai.dueros.device_interface.extensions.telephone",
      "name": "PhonecallByNumber",
      "messageId": "{{STRING}}",
      "dialogRequestId": "{{STRING}}"
    },
    "payload": {
      "number": "{{STRING}}"
    }
  }
}
```



```
"payload": {
  "publicServiceName": "{{STRING}}",
  "phoneNumber": "{{STRING}}"
}
```

Payload 参数说明

- publicServiceName
 - 公用服务号码, eg: 10086、110、120 ...
- phoneNumber
 - 联系人手机号

3. SendSmsByName 指令

向通讯录的联系人发短信

消息样例

```
json
"directive": {
  "header": {
    "namespace": "ai.dueros.device_interface.extensions.sms",
    "name": "SendSmsByName",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "name": "{{STRING}}",
    "content": "{{STRING}}"
  }
}
```

Payload 参数说明

- name
 - 联系人名称
- content
 - 短信内容

4. SendSmsByNumber 指令

向唯一的电话号码发送短信

消息样例

```
json
{
  "directive": {
    "header": {
      "namespace": "ai.dueros.device_interface.extensions.sms",
      "name": "SendSmsByNumber",
      "messageId": "{{STRING}}",
      "dialogRequestId": "{{STRING}}"
    },
    "payload": {
      "phoneNumber": "{{STRING}}",
      "content": "{{STRING}}"
    }
  }
}
```

Payload 参数说明

- phoneNumber
 - 联系人手机号
- content
 - 短信内容

地图导航

1. Navigator 指令

地图导航的通用协议指令

消息样例

```
json
{
  "directive": {
    "header": {
      "namespace":
"ai.dueros.device_interface.extensions.navigation",
```

```

        "name": "Navigator",
        "messageId": "{{STRING}}",
        "dialogRequestId": "{{STRING}}"
    },
    "payload": {
        "action": "{{ENUM}}",
        "place": "{{STRING}}"
    }
}

```

Payload 参数说明

- action
 - 导航指令，Start：开始导航、AddStop：添加途经点、RemoveStop：取消途经点、Continue：继续导航、SwitchRoad：切换路线、EndTrip：导航结束
- place
 - 途经点地址，eg：途径合肥

2. SwitchRoutePreference 指令

路线偏好指令协议

消息样例

```

json
{
  "directive": {
    "header": {
      "namespace":
"ai.dueros.device_interface.extensions.navigation",
      "name": "SwitchRoutePreference",
      "messageId": "{{STRING}}",
      "dialogRequestId": "{{STRING}}"
    },
    "payload": {
      "preference": "{{ENUM}}"
    }
  }
}

```

Payload 参数说明

- preference

- 路线偏好, AvoidElevatedHighway: 避开高架、AvoidHighway: 避开高速、NationalRoadPriority: 国道优先

3. Search 指令

导航搜索指令

消息样例

```
json
{
  "directive": {
    "header": {
      "namespace":
        "ai.dueros.device_interface.extensions.navigation",
      "name": "Search",
      "messageId": "{{STRING}}",
      "dialogRequestId": "{{STRING}}"
    },
    "payload": {
      "target": "{{ENUM}}",
      "place": "{{STRING}}"
    }
  }
}
```

Payload 参数说明

- target
 - 询问目标, Distance: 距离、Duration: 时间、SpeedLimit: 限速
- place
 - 目的地地点, eg: 距离深圳机场有多远

4. Function 指令

导航功能相关指令

消息样例

```
json
{
  "directive": {
    "header": {
      "namespace":
```

```

"ai.dueros.device_interface.extensions.navigation",
  "name": "Function",
  "messageId": "{{STRING}}",
  "dialogRequestId": "{{STRING}}"
},
"payload": {
  "action": "{{ENUM}}",
  "function": "{{ENUM}}"
}
}

```

Payload 参数说明

- action
 - 是否启用功能，EnableFunction：启用/打开、DisableFunction：禁用/关闭
- function
 - 功能名称，NaviBroadcast：导航播报

5. NavigateTo 指令

导航到目的地的协议指令

消息样例

```

json
"directive": {
  "header": {
    "namespace":
"ai.dueros.device_interface.extensions.navigation",
    "name": "NavigateTo",
    "messageId": "{{STRING}}",
    "dialogRequestId": "{{STRING}}"
  },
  "payload": {
    "place": "{{STRING}}"
  }
}

```

Payload 参数说明

- place

- 目的地地点，eg：深圳机场

文生图指令

1. RenderProcessing 指令

图片生成进度

消息样例

```
json
{
  "directive": {
    "header": {
      "name": "RenderProcessing",
      "namespace": "ai.dueros.device_interface.screen"
    },
    "payload": {
      "desc": "{{STRING}}",
      "percent": {{INT}}
    }
  }
}
```

Payload 参数说明

- desc
 - 生图描述
- percent
 - 生成百分比

2. RenderMultiImageCard 指令

图片生成结果

消息样例

```
json
{
  "directive": {
    "header": {
      "name": "RenderMultiImageCard",
      "namespace": "ai.dueros.device_interface.screen"
    }
  }
}
```

```
    },
    "payload": {
      "desc": "{{STRING}}",
      "images": [
        {
          "url": "{{STRING}}",
          "share_url": "{{STRING}}"
        }
      ]
    }
  }
}
```

Payload 参数说明

- desc
 - 图片描述
- Images
 - url 图片链接(出于安全考虑, 一个小时后禁止访问图片)
 - share_url 图片分享链接, 持久化存储